AWS ARCHITECT ASSOCIATE / EWB CLOUD LABS

VPC Peering

Project 01: Connect two VPCs. Prove private connectivity.

Name: Aryan Rao

Batch: Batch 1

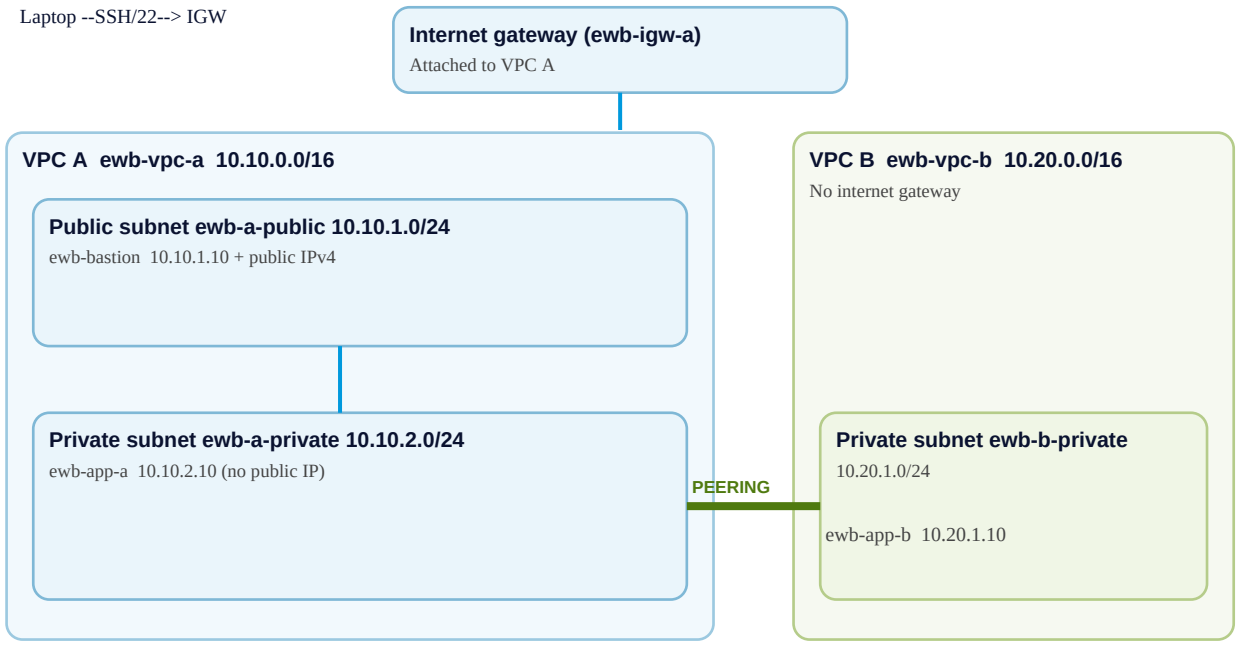
Region: Asia Pacific (Mumbai) / ap-south-1

Completion date: 24 October 2026

1. Project overview

This project connects two private VPCs using VPC peering so that two application servers can exchange traffic using private IPv4 addresses only. One public bastion host in VPC A is used for SSH administration through ProxyJump. No NAT gateway is used and neither application server has a public IP. Everything was built in one account and one Region (Mumbai, ap-south-1) using the AWS Console, and tested from Windows PowerShell with OpenSSH.

2. Architecture diagram



Administration path: laptop > bastion > App-A or App-B using SSH ProxyJump. Project traffic: App-A <-> App-B over private IPs through the peering connection.

3. Resource inventory

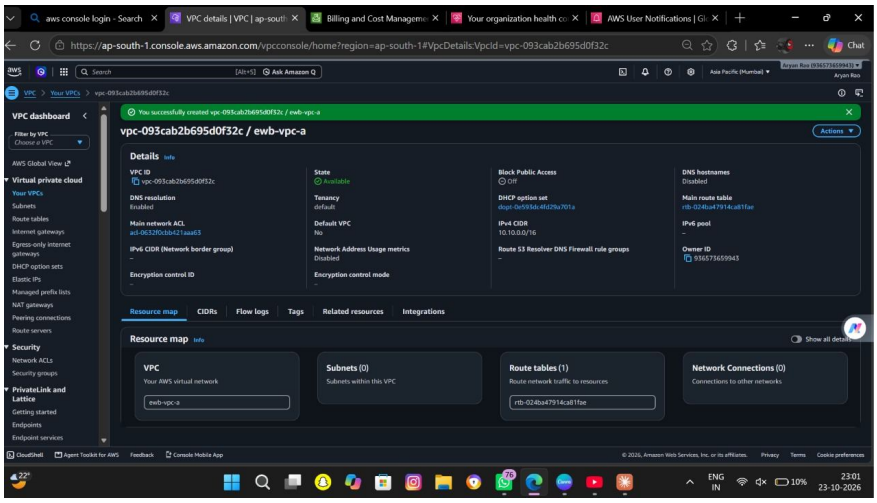
Resource	Name tag	Resource ID	CIDR / IP
VPC	ewb-vpc-a	vpc-093cab2b695d0f32c	10.10.0.0/16
Subnet	ewb-a-public	subnet-069e3cdabd530521c	10.10.1.0/24
Subnet	ewb-a-private	subnet-09d9b92dd5029595e	10.10.2.0/24
VPC	ewb-vpc-b	vpc-035e8528d5cb89361	10.20.0.0/16
Subnet	ewb-b-private	subnet-0a11d64ae0dc324c0	10.20.1.0/24
Internet gateway	ewb-igw-a	igw-0b7b9d13a2509d7e9	Attached to VPC A
Peering	ewb-a-b-peer	pcx-0872ea58430aa0767	Active
Route table	ewb-rt-a-public	rtb-0febf1ed1f56ed3b2	ewb-a-public
Route table	ewb-rt-a-private	rtb-0cdccd9abe755fa1e	ewb-a-private
Route table	ewb-rt-b-private	rtb-089a4d61270453b3f	ewb-b-private
Security group	ewb-sg-bastion	sg-0e0e5f0d00898014b	VPC A
Security group	ewb-sg-app-a	sg-02061d39b7b6dd527	VPC A
Security group	ewb-sg-app-b	sg-0c52323faee935054	VPC B
EC2	ewb-bastion	i-0fba8d3dfd887f8e0	10.10.1.10 + public 13.207.201.119
EC2	ewb-app-a	i-0bebecc6d8d70d21d	10.10.2.10 (no public IP)

Resource	Name tag	Resource ID	CIDR / IP
EC2	ewb-app-b	i-0eff68cbd4536e0fb	10.20.1.10 (no public IP)

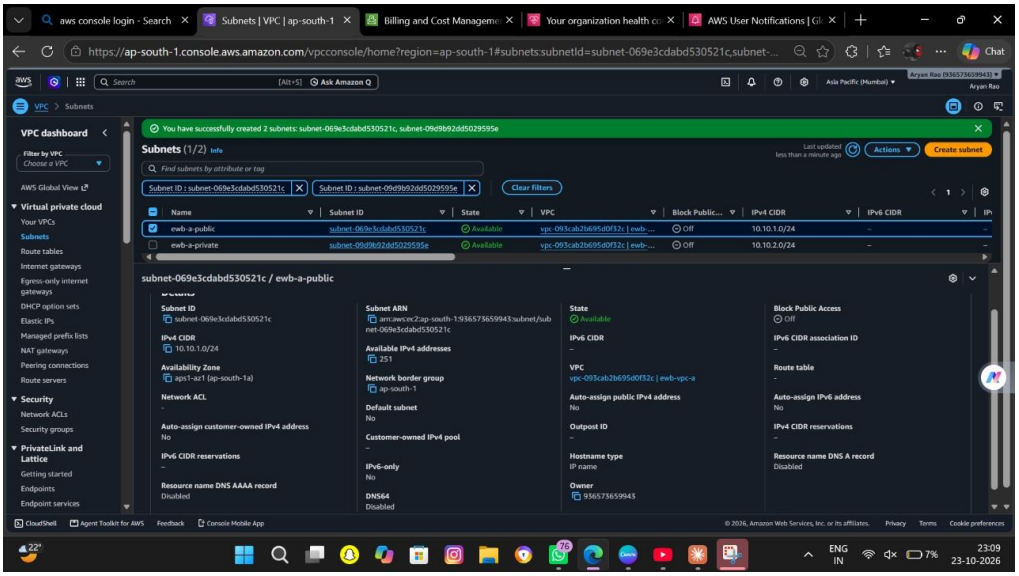
CIDR planning: 10.10.0.0/16 and 10.20.0.0/16 do not overlap, which is required for peering. Each /24 subnet sits inside its own VPC range.

4. Network build: VPCs, subnets, gateway

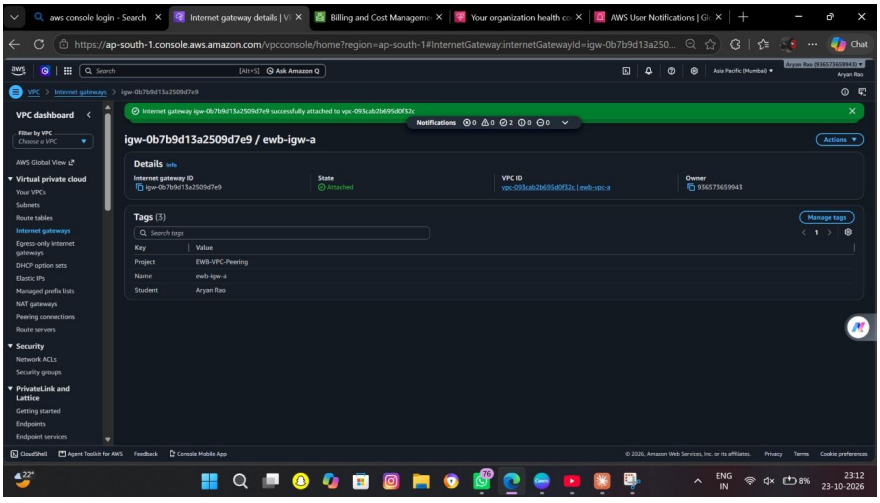
VPC A: ewb-vpc-a, 10.10.0.0/16, Available



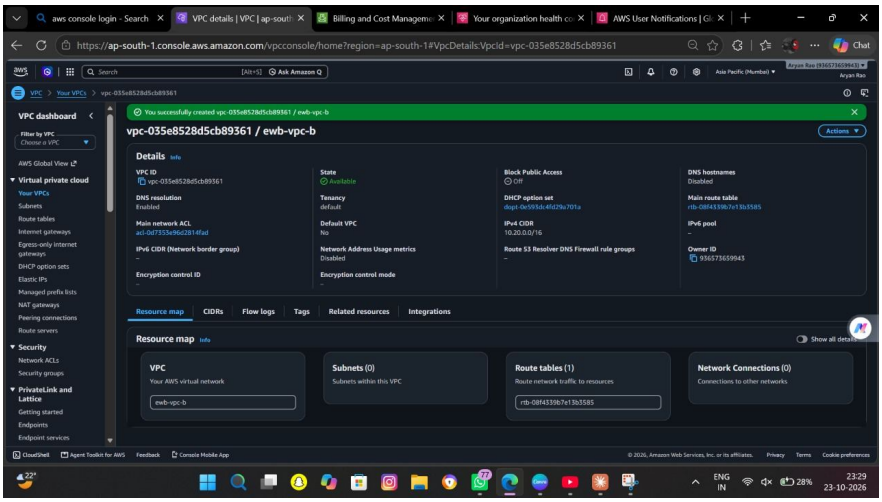
VPC A subnets: ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24) in ap-south-1a



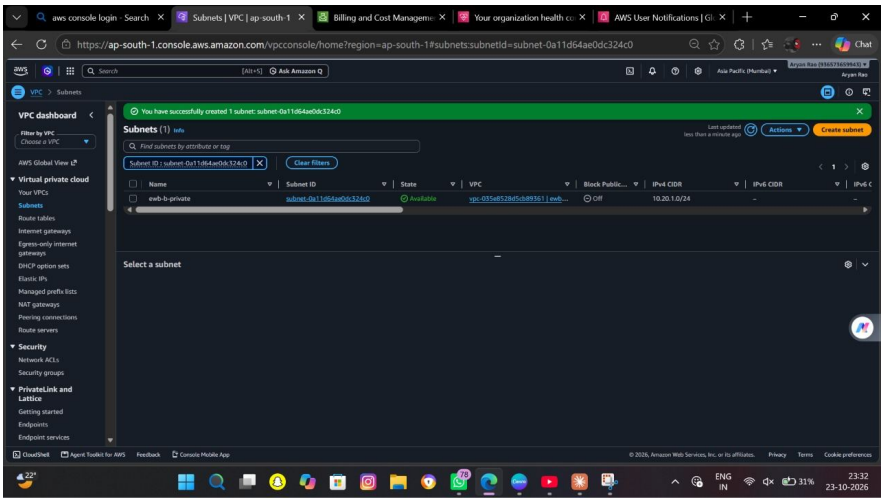
Internet gateway ewb-igw-a attached to ewb-vpc-a



VPC B: ewb-vpc-b, 10.20.0.0/16, Available



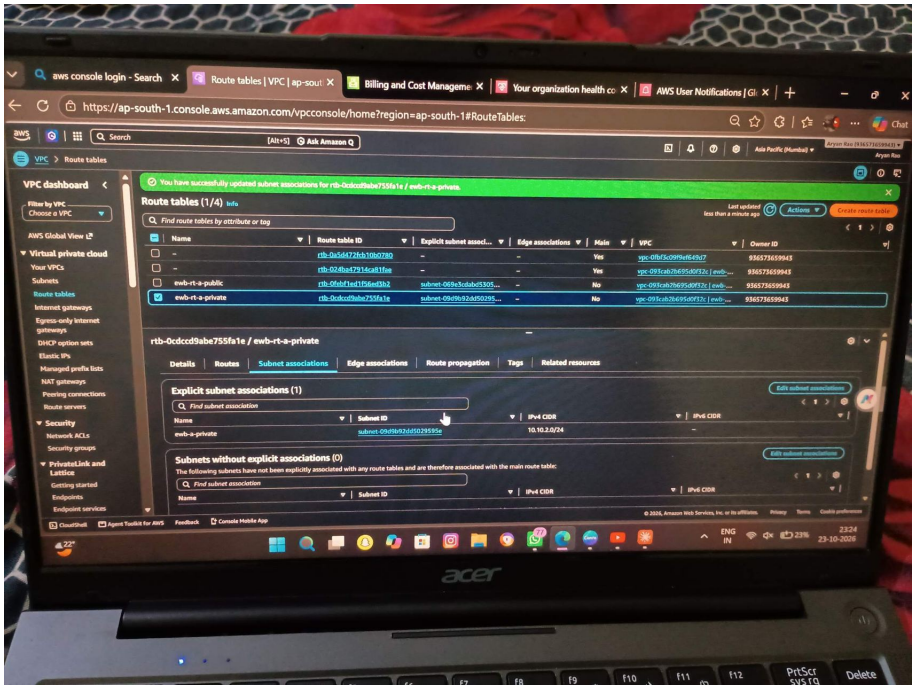
VPC B subnet: ewb-b-private, 10.20.1.0/24



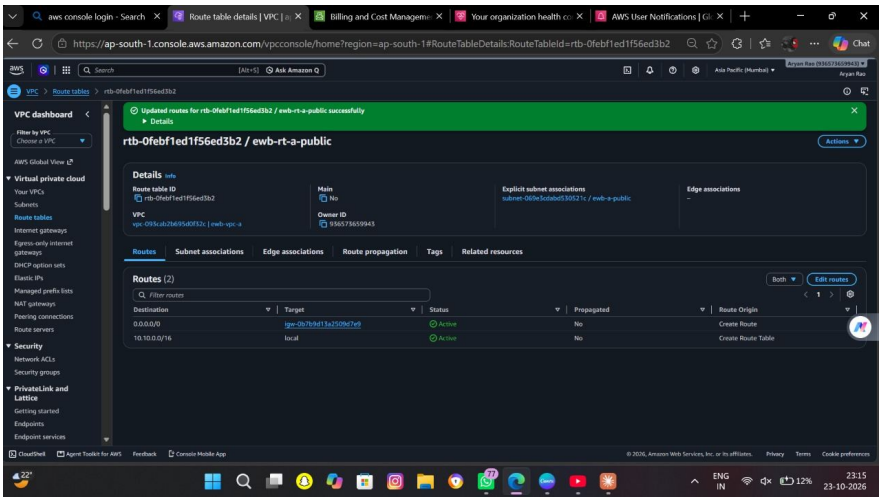
5. Route tables and subnet associations

Route table	Associated subnet	Destination	Target
ewb-rt-a-public	ewb-a-public	10.10.0.0/16	local
		0.0.0.0/0	igw-0b7b9d13a2509d7e9
		10.20.0.0/16	pcx-0872ea58430aa0767
ewb-rt-a-private	ewb-a-private	10.10.0.0/16	local
		10.20.0.0/16	pcx-0872ea58430aa0767
ewb-rt-b-private	ewb-b-private	10.20.0.0/16	local
		10.10.0.0/16	pcx-0872ea58430aa0767

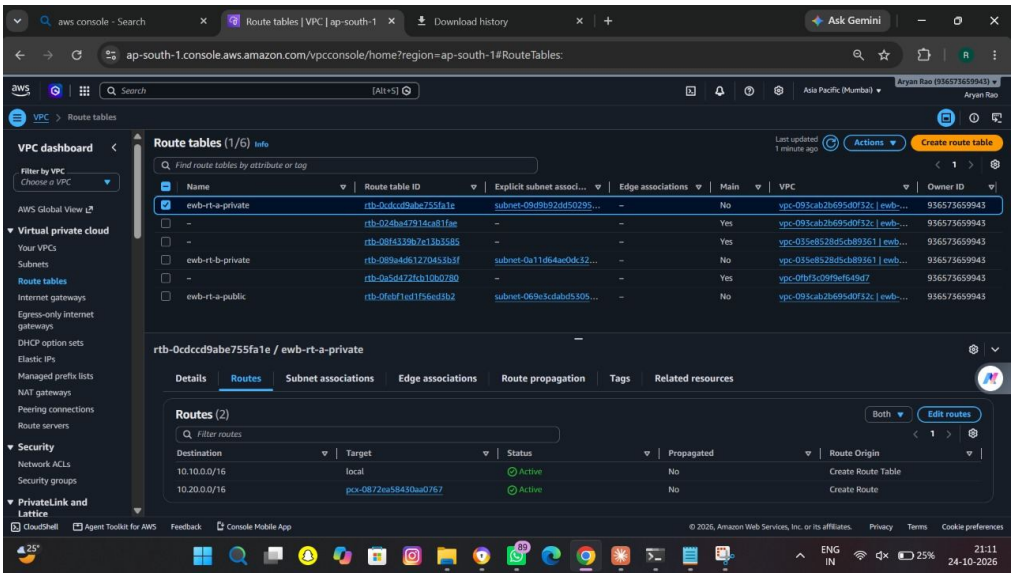
Checkpoint 01: explicit subnet associations. ewb-rt-a-private is selected; ewb-rt-a-public is associated with ewb-a-public



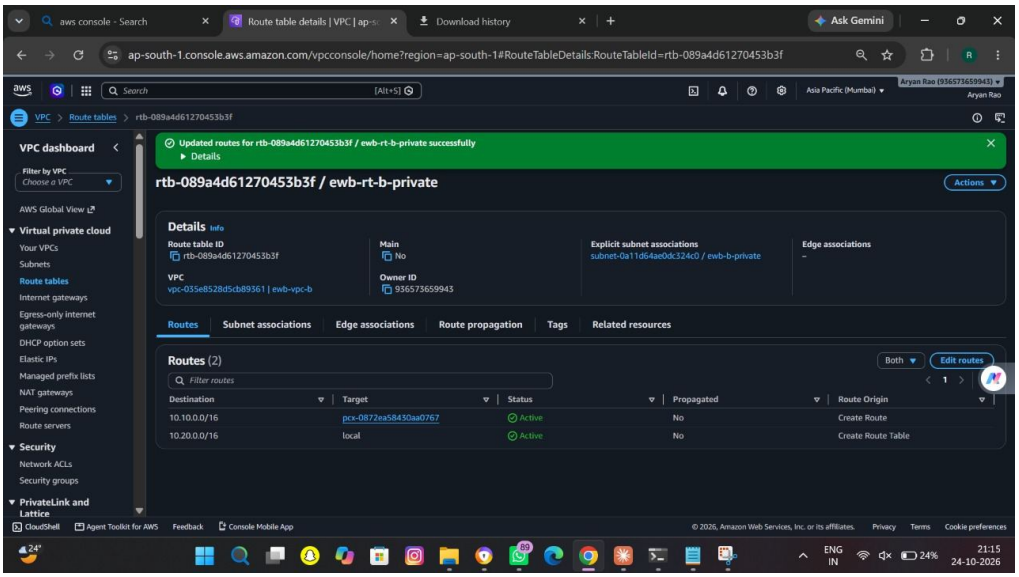
ewb-rt-a-public: public default route to the internet gateway (before peering)



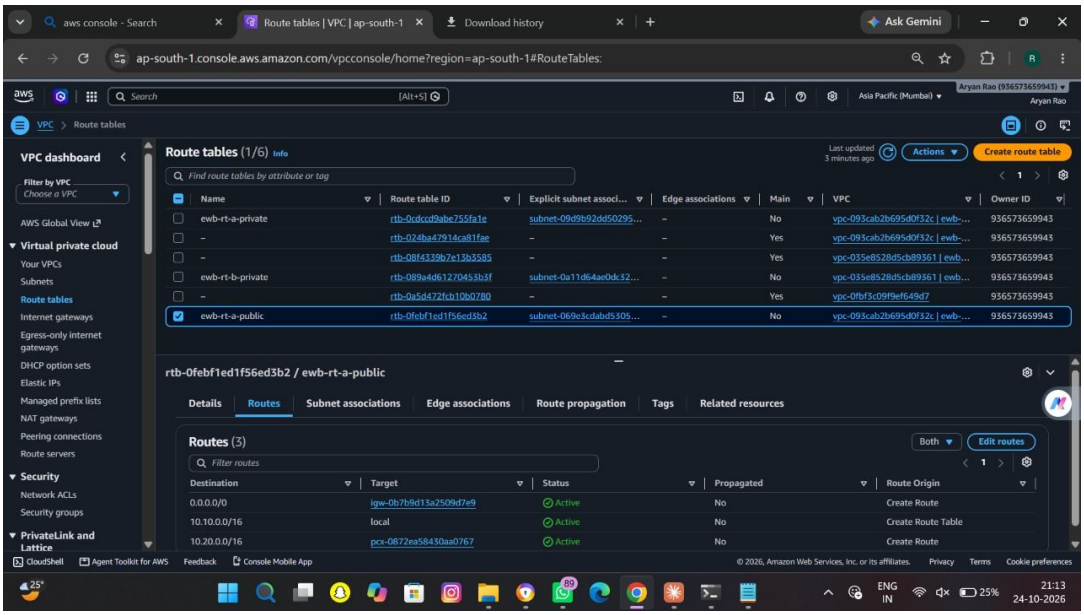
Checkpoint 04: ewb-rt-a-private routes, 10.20.0.0/16 to pcx-0872ea58430aa0767 (Active)



Checkpoint 04: ewb-rt-b-private routes, return path 10.10.0.0/16 to pcx-0872ea58430aa0767 (Active)



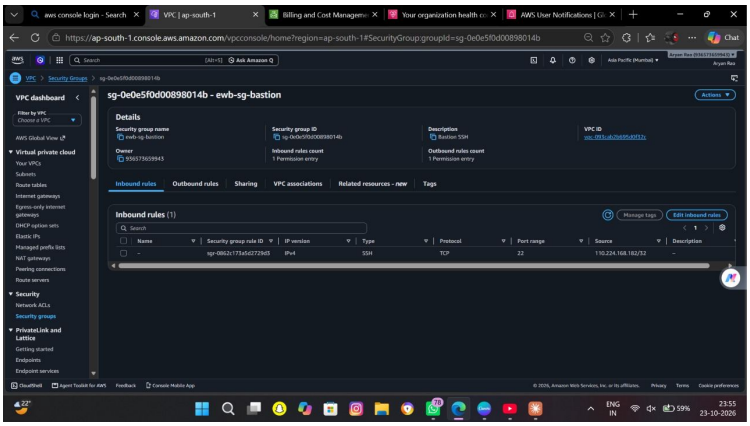
Checkpoint 04: ewb-rt-a-public routes: local, 0.0.0.0/0 to IGW, 10.20.0.0/16 to pcx (Active)



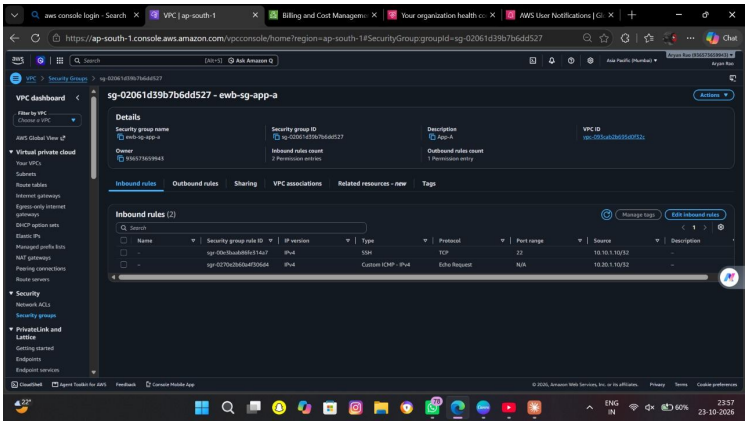
6. Security groups

Security group	VPC	Type / port	Source
ewb-sg-bastion	A	SSH / TCP 22	My public IPv4 /32
ewb-sg-app-a	A	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a	A	ICMP echo request (type 8)	10.20.1.10/32 (App-B)
ewb-sg-app-b	B	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b	B	ICMP echo request (type 8)	10.10.2.10/32 (App-A)

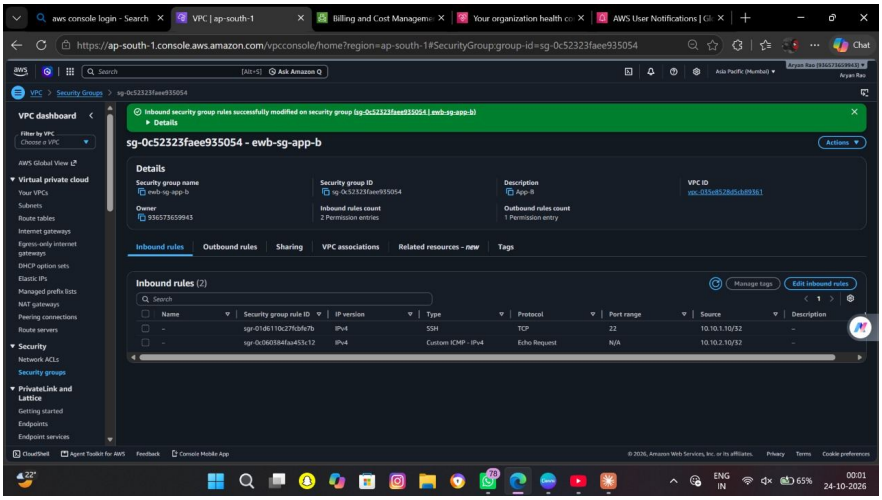
ewb-sg-bastion inbound rules



ewb-sg-app-a inbound rules

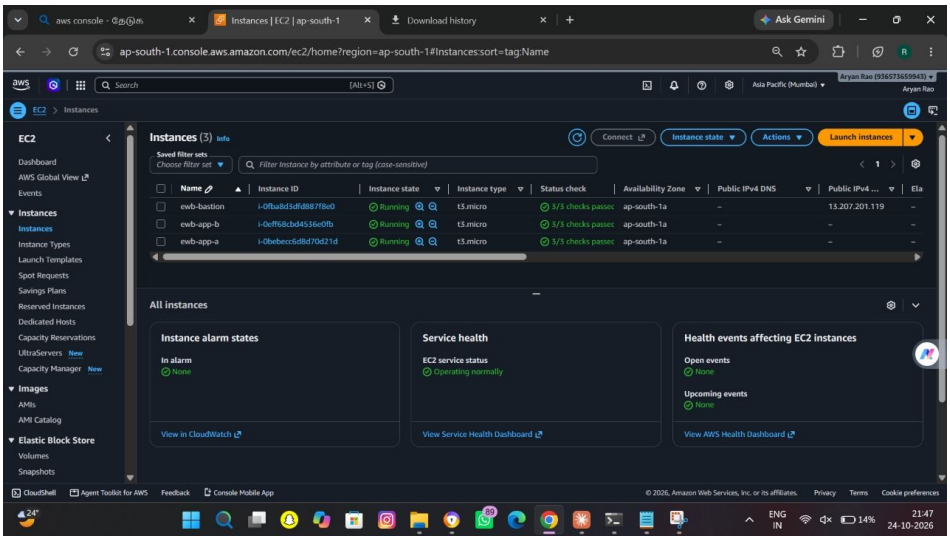


ewb-sg-app-b inbound rules (SSH from bastion, ICMP echo from App-A)

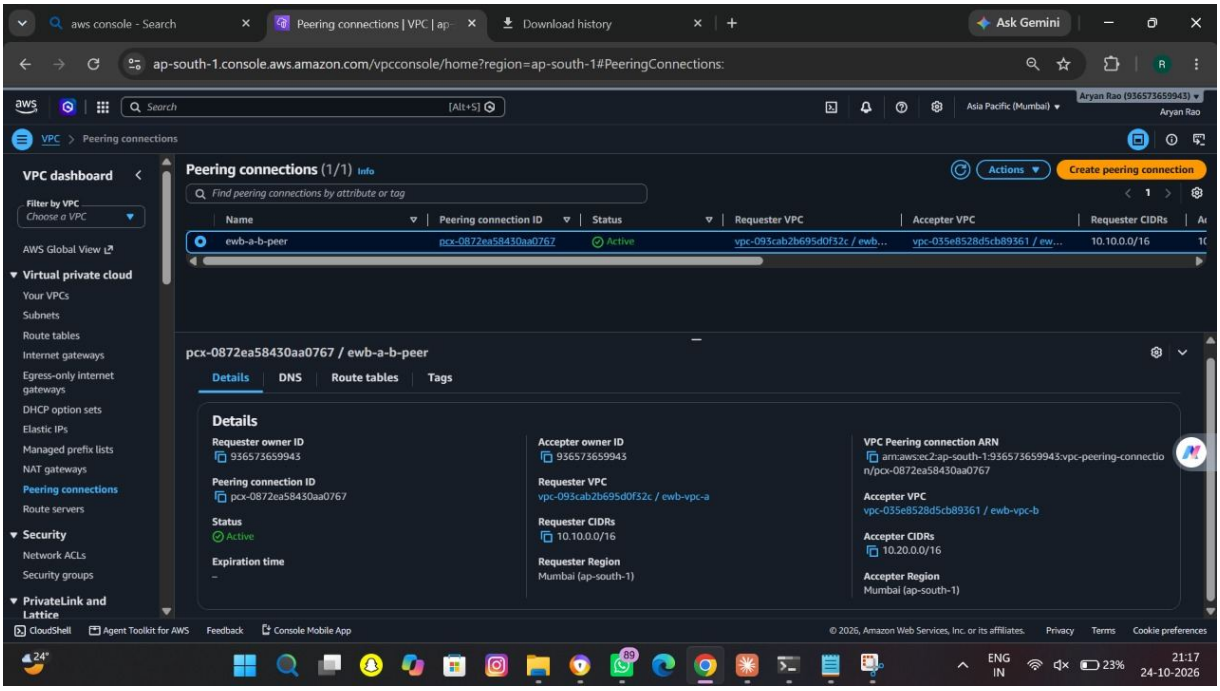


7. Instances and peering connection

Checkpoint 02: ewb-bastion, ewb-app-a and ewb-app-b running, 3/3 checks passed. Only the bastion has a public IPv4



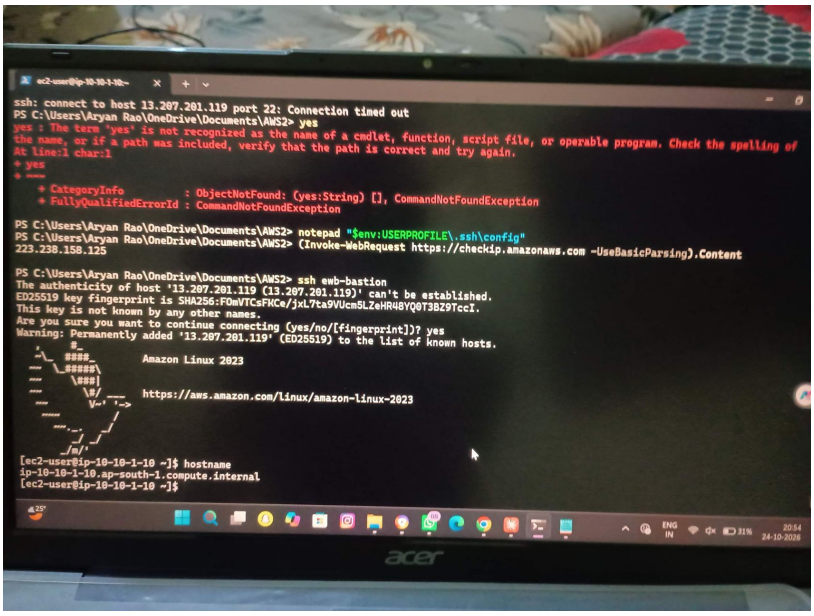
Checkpoint 03: ewb-a-b-peer is Active. Requester ewb-vpc-a (10.10.0.0/16), accepter ewb-vpc-b (10.20.0.0/16)



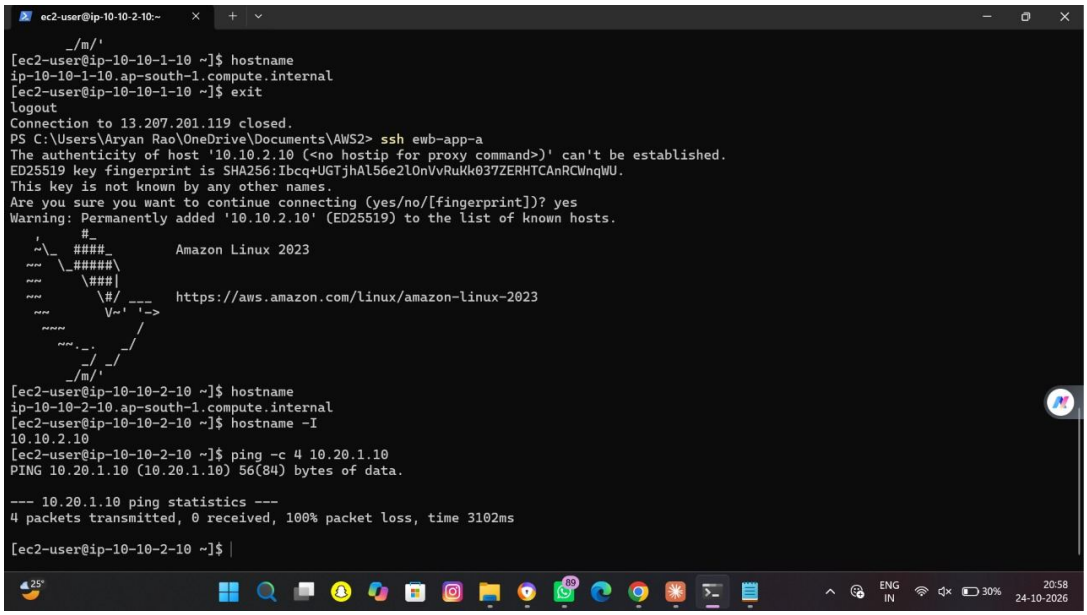
8. Test results

Test	Where / command	Expected	Result
01 Baseline	App-A before peering: ping -c 4 10.20.1.10	No replies	4 transmitted, 0 received, 100% packet loss
02 Forward	App-A after routes: ping -c 4 10.20.1.10	0% loss	4 received, 0% packet loss
03 Reverse	App-B: ping -c 4 10.10.2.10	0% loss	4 received, 0% packet loss
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Connected, hostname ip-10-20-1-10
05 Isolation	EC2 details of App-A and App-B	No public IPv4	Public IPv4 empty on both

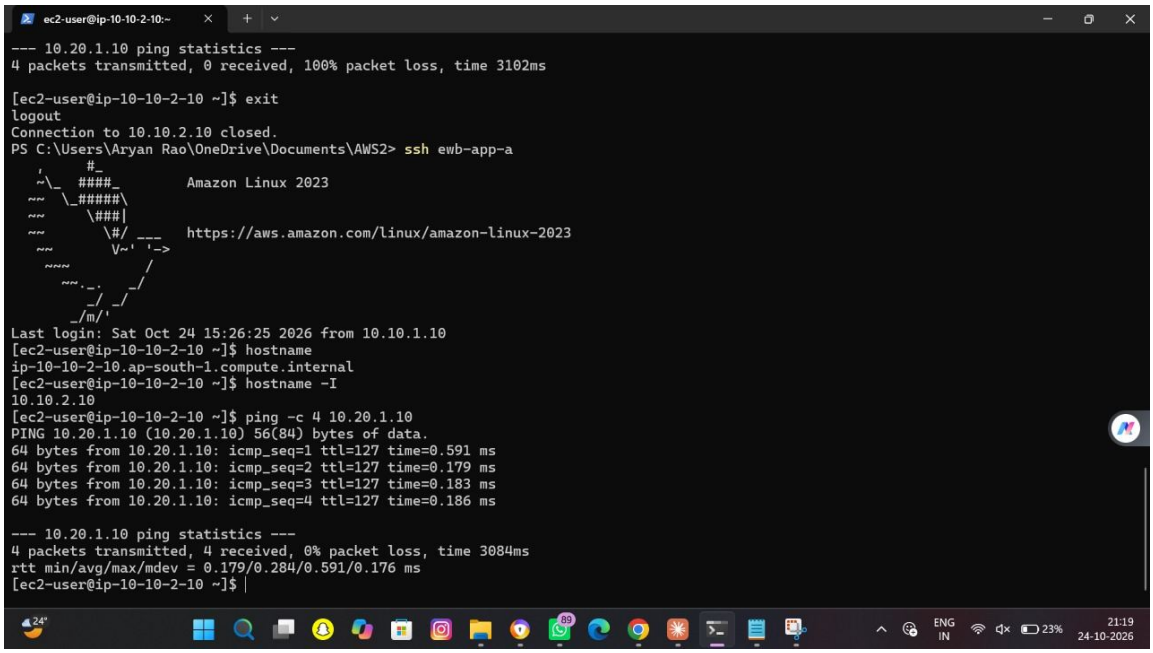
Bastion SSH from laptop: Amazon Linux 2023, hostname ip-10-10-1-10 (public IP 13.207.201.119)



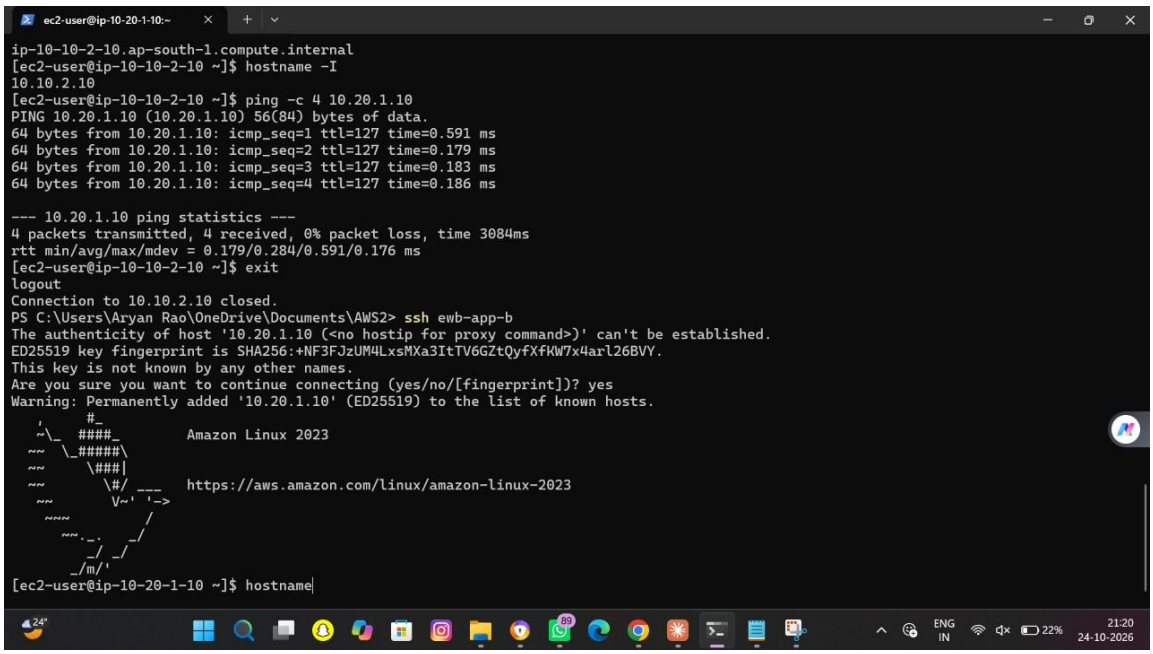
Test 01 / App-A / Baseline before peering: ping to 10.20.1.10 fails, 100% packet loss



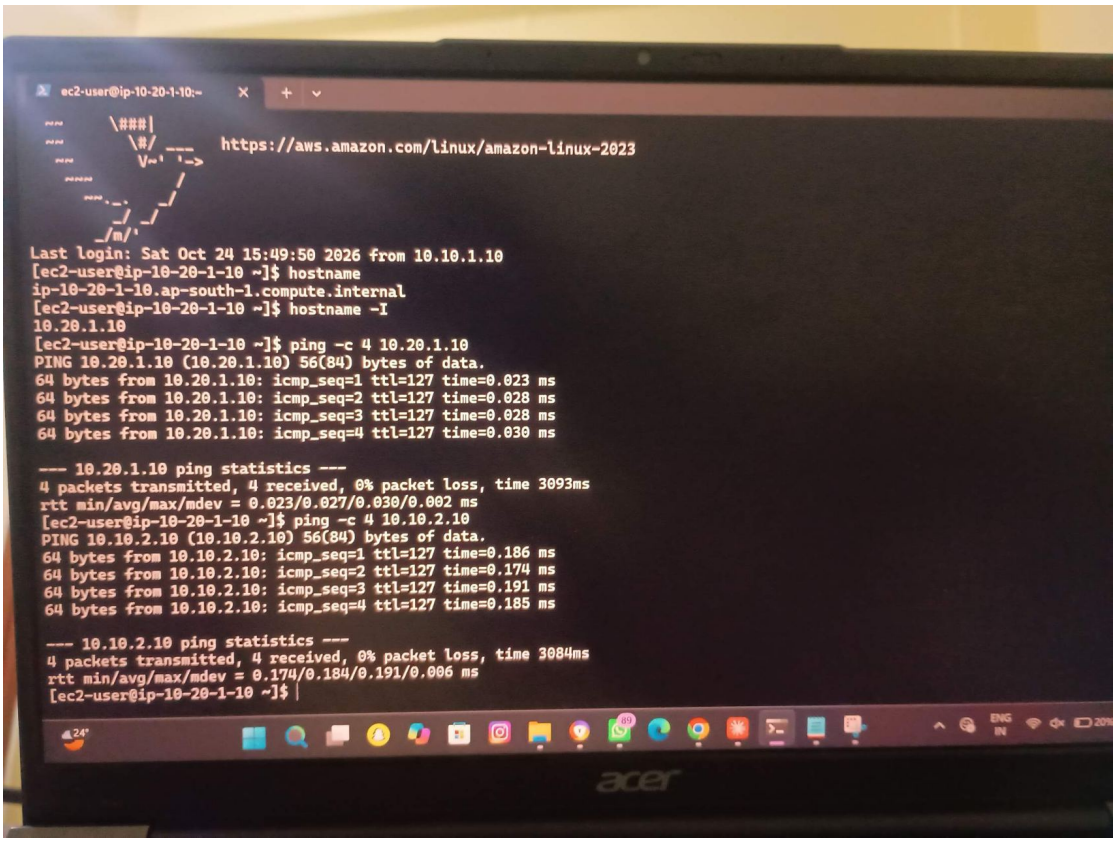
Test 02 / App-A (10.10.2.10) / Forward ping to 10.20.1.10 succeeds after peering and routes, 0% loss



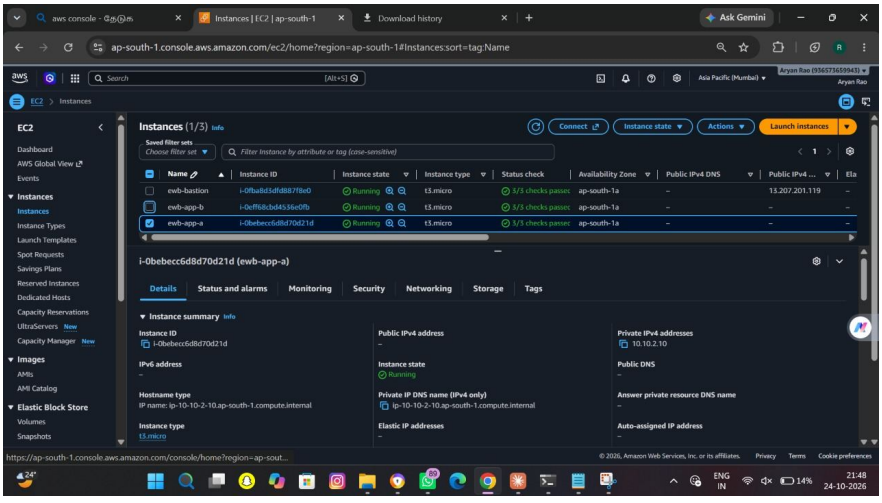
Test 04 / Laptop / ssh ewb-app-b through the bastion (ip-10-20-1-10)



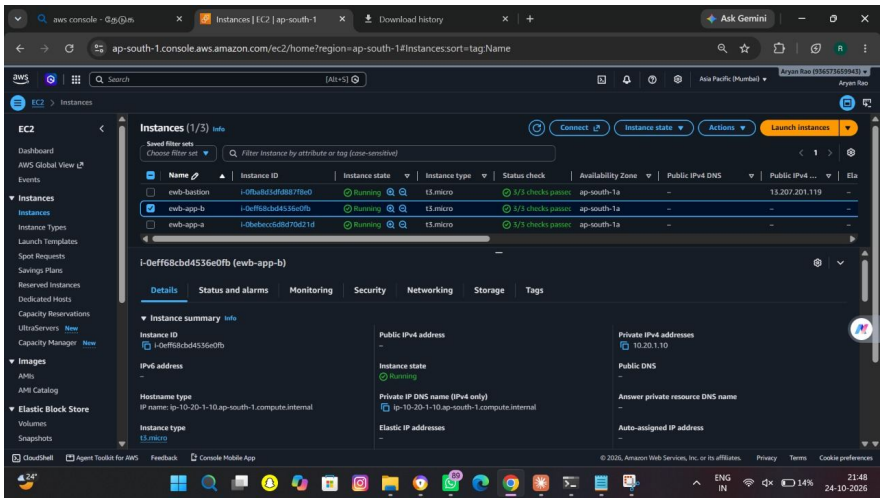
Test 03 / App-B (10.20.1.10) / Reverse ping to 10.10.2.10 succeeds, 0% loss (second command)



Test 05 / App-A: Private IPv4 10.10.2.10, Public IPv4 empty



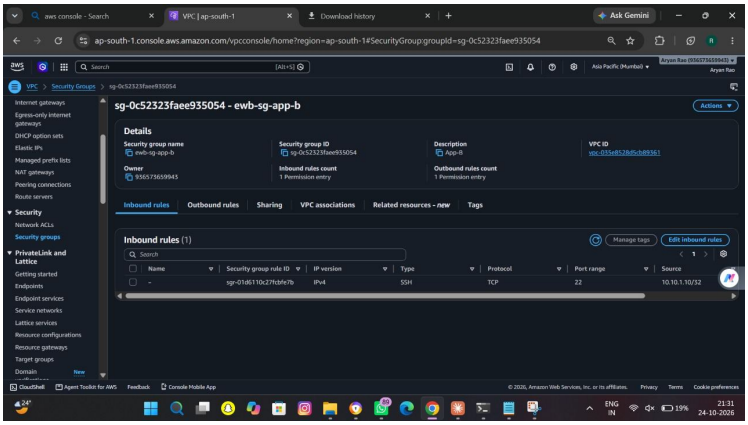
Test 05 / App-B: Private IPv4 10.20.1.10, Public IPv4 empty



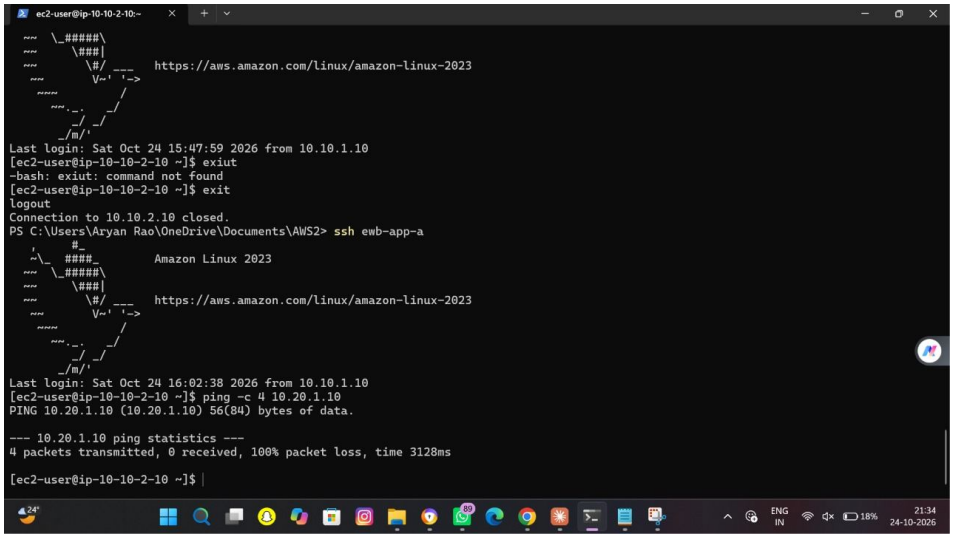
9. Controlled failure and recovery

I removed only the inbound ICMP echo request rule on ewb-sg-app-b (source 10.10.2.10/32) and kept the SSH rule. The forward ping from App-A stopped getting replies. After adding the exact same rule back, replies resumed. This shows that the destination security group controls whether traffic can use the working peering path.

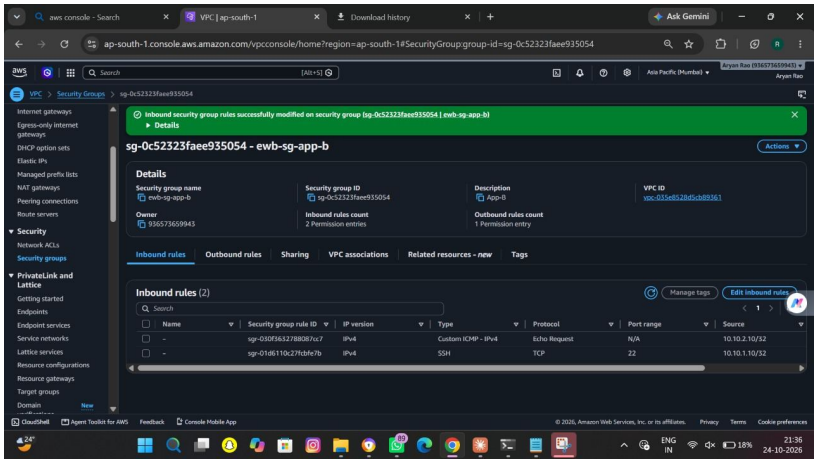
Failure step 1: ewb-sg-app-b has only the SSH rule



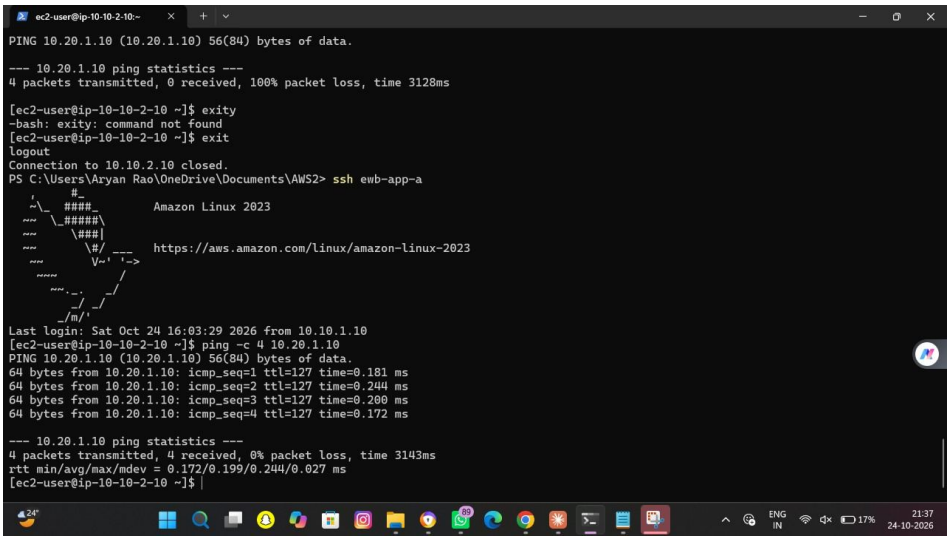
Failure step 2 / App-A: ping to 10.20.1.10, 100% packet loss



Recovery step 1: ICMP echo request rule restored on ewb-sg-app-b



Recovery step 2 / App-A: ping to 10.20.1.10, 0% packet loss



10. Forward path and return path

Forward path: App-A (10.10.2.10) sends a packet to 10.20.1.10. The route table ewb-rt-a-private contains 10.20.0.0/16 > pcx, so the packet goes through the peering connection into VPC B. The security group ewb-sg-app-b allows ICMP echo request from 10.10.2.10/32, so App-B accepts it.

Return path: App-B replies to 10.10.2.10. The route table ewb-rt-b-private contains 10.10.0.0/16 > pcx, so the reply goes back through the peering connection. Security groups are stateful, so the reply is allowed automatically.

11. Questions and answers

1. Why is the return route needed?
Routing works in each direction separately. Without a route in VPC B for 10.10.0.0/16 pointing to the peering connection, App-B would not know how to send replies, so ping and SSH would fail.

2. Why is the App-B SSH source the bastion private IP?
With ProxyJump, the bastion opens a new SSH connection to App-B. App-B sees the bastion private IP 10.10.1.10 as the source, not my laptop IP.

3. Why does Active peering not guarantee a successful ping?
Peering is only the link. Traffic also needs routes in both VPCs, security group rules on the destination, and permissive network ACLs. In this project the failure test showed that removing the ICMP rule stops ping even though peering stays Active.

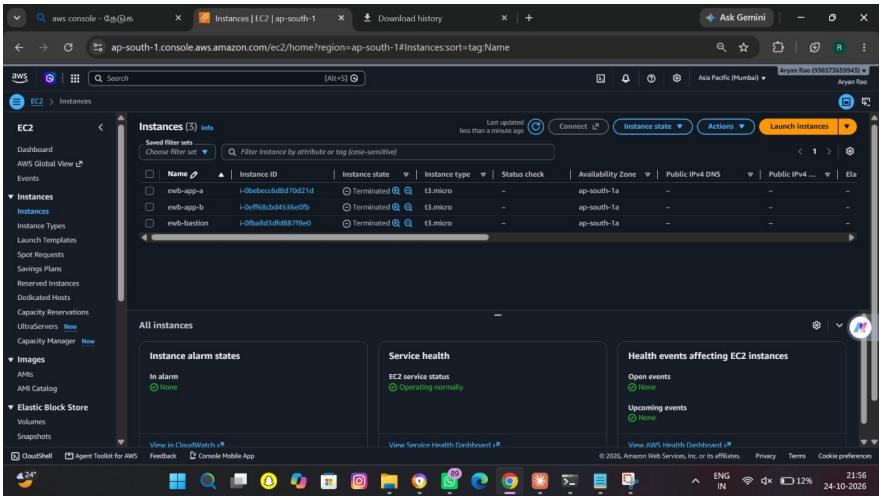
4. Can VPC B use VPC A's internet gateway over peering?
No. VPC peering is non-transitive and does not allow routing through the other VPC's internet gateway. VPC B stays without internet access.

Why does the public subnet need a peering route?
The bastion itself starts the SSH connection to App-B, so its subnet must know how to reach 10.20.0.0/16. The more specific 10.20.0.0/16 route is chosen over the 0.0.0.0/0 internet route.

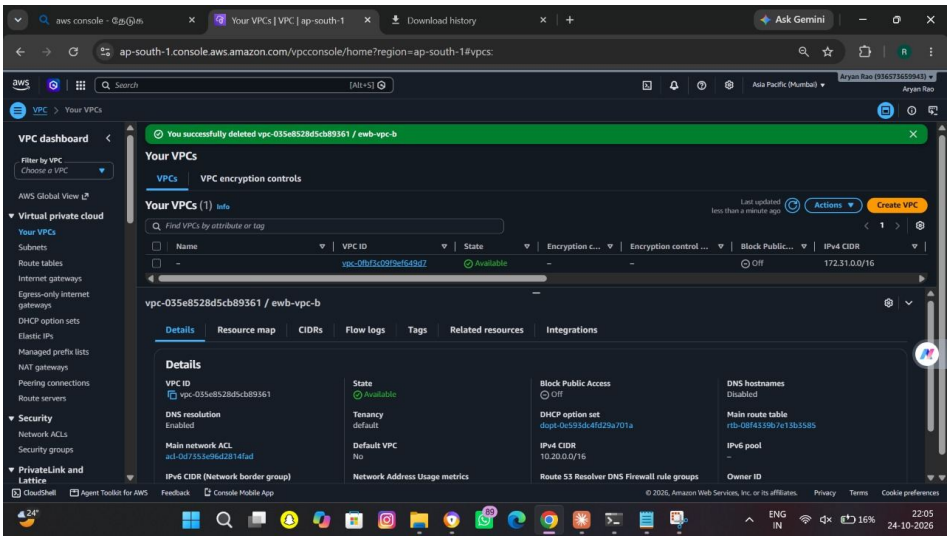
12. Cleanup

After saving all evidence, the instances, peering connection, security groups, route tables, subnets, internet gateway, both VPCs and the key pair are deleted. Cleanup evidence is below: all instances show Terminated and only the default VPC is left.

Cleanup 1: ewb-app-a, ewb-app-b and ewb-bastion are Terminated



Cleanup 2: ewb-vpc-b deleted; only the default VPC (172.31.0.0/16) is left in Your VPCs



13. What I learned

In this project I learned the basics of cloud computing networking on AWS. I understood how VPCs, subnets and route tables work together, and why CIDR ranges must not overlap for peering. I learned that peering needs routes on both sides and correct security group rules, and that a bastion host with SSH ProxyJump lets me manage private servers without giving them public IPs. I also learned to test, break and fix a setup, to read my own mistakes (a mistyped route CIDR and swapped Name tags), and to clean up resources to avoid extra cost.